



Kauno technologijos universitetas

Elektros ir elektronikos fakultetas

Jutiklių duomenų vizualizavimas

T170B008 Kompiuterinės komunikacijos

Laboratorinio darbo Nr. 2 ataskaita

Linas Beinorius

Studentas

Dr. Pranas Kuzas

Dėstytojas

Kaunas, 2025

Turinys

Turinys.....	2
Paveikslų sąrašas.....	3
Darbo tikslas.....	4
1. Darbo eiga ir rezultatai.....	5
1.1. Parametrų parinkimo langas.....	5
1.2. Globalių kintamųjų failas.....	7
1.3. Funkcijų failas.....	8
1.4. Pagrindinis failas.....	12
Išvados.....	17
Šaltiniai.....	18

Paveikslų sąrašas

1 pav.	Siunčiamų duomenų paketas.....	4
2 pav.	Parametrų nustatymo langas.....	5
3 pav.	Grafinės vartotojo sąsajos pradinis vaizdas.....	13
4 pav.	Nuoseklaus prievado iššokantis konfigūravimo langas.....	14
5 pav.	Duomenų vizualizavimas vartotojo sąsajoje.....	14
6 pav.	Duomenų išsaugojimas tekstiniame faile „csv“ formatu.....	15
7 pav.	Programos metu naudojami failai.....	16

Darbo tikslas

Atvaizduoti per UART sąsają priimamus duomenis susikurtoje grafinėje vartotojo sąsajoje.

Duomenys yra pateikiami pagal pirmame laboratoriniame darbe sudarytą paketą.

```
$PNLBGPS,1,54.53.35.07,N,24.00.43.51,E,140250,7,0.0,*54
$PNLBLPS,1011.9,26.4,*75
$PNLBLPS,1011.9,26.4,*75
$PNLBLPS,1011.9,26.4,*75
$PNLBGPS,1,54.53.34.98,N,24.00.43.78,E,140251,8,0.0,*56
$PNLBLPS,1011.9,26.4,*75
$PNLBLPS,1011.8,26.4,*74
$PNLBLPS,1011.9,26.4,*75
$PNLBGPS,1,54.53.34.88,N,24.00.43.98,E,140252,7,0.0,*55
$PNLBLPS,1011.8,26.4,*74
$PNLBLPS,1011.9,26.4,*75
$PNLBLPS,1011.9,26.4,*75
$PNLBGPS,1,54.53.34.81,N,24.00.44.11,E,140253,7,0.0,*5B
$PNLBLPS,1011.8,26.4,*74
$PNLBLPS,1011.8,26.4,*74
$PNLBLPS,1011.9,26.4,*75
$PNLBGPS,1,54.53.34.76,N,24.00.44.20,E,140254,7,0.0,*56
$PNLBLPS,1011.9,26.4,*75
$PNLBLPS,1011.9,27.1,*71
$PNLBLPS,1011.9,27.7,*77
$PNLBGPS,1,54.53.34.72,N,24.00.44.30,E,140255,7,0.0,*52
$PNLBLPS,1011.9,28.1,*7E
$PNLBLPS,1011.9,28.4,*7B
$PNLBLPS,1011.9,28.8,*77
$PNLBGPS,1,54.53.34.73,N,24.00.44.36,E,140256,8,0.0,*59
$PNLBLPS,1011.9,29.1,*7F
$PNLBLPS,1011.9,28.9,*76
$PNLBLPS,1011.9,28.3,*7C
$PNLBGPS,1,54.53.34.72,N,24.00.44.43,E,140257,7,0.0,*54
$PNLBLPS,1011.9,27.8,*78
$PNLBLPS,1011.9,27.6,*76
$PNLBLPS,1011.9,27.5,*75
$PNLBGPS,1,54.53.34.68,N,24.00.44.50,E,140258,5,1.0,*51
$PNLBLPS,1011.9,27.4,*74
$PNLBLPS,1011.9,27.4,*74
$PNLBLPS,1011.8,27.3,*72
$PNLBGPS,1,54.53.34.69,N,24.00.44.48,E,140259,6,1.0,*5B
$PNLBLPS,1011.9,27.3,*73
```

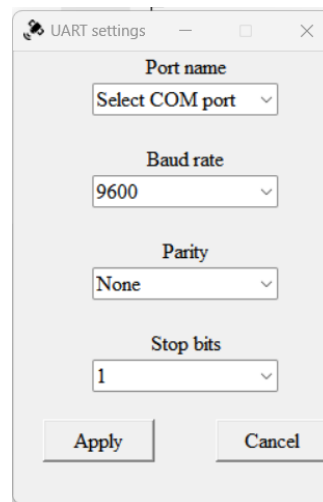
1 pav. Siunčiamų duomenų paketas

LPS22HB jutiklio duomenys yra perduodami per atskirą komandą, kuri prasideda \$PNLBLPS. Vertės yra atskiriamos kableliais. Pirmas duomuo yra slėgis hektopaskaliais, po to seka temperatūra °C ir komanda baigiasi kontroline suma. Kontrolinė suma yra viso paketo XOR vertė ir ji yra aprašoma „ASCII“ vertėmis kaip šešiolyktainė vertė (55 reiškia 0x55 kontrolinė suma). GPS duomenys yra perduodami per \$PNLBGPS paketą. Pirmą vertę nusako GNSS modulio padėties užfiksavimą, po to seka ilguma DMS formatu ir kryptis, platumą DMS formatu ir jos kryptis, ETC vasaros laikas, palydovų susisiekimo skaičius, HDOP vertė ir baigiasi kontroline suma. Jutiklių duomenys yra perduodami 1/3 sekundės periodu, o GPS duomenys kas vieną sekundę.

1. Darbo eiga ir rezultatai

Grafinė vartotojo sąsaja yra rašoma VS Code redaktoriuje pasinaudojant „tkinter“ paketu. Kodas yra rašomas nuo pradžios, todėl reikalinga iš pirmo bendrai apsisąstyti, kaip turėtų atrodyti grafinė sąsaja ir aprašyti funkcionalumą. Visų pirma yra reikalinga nuoseklaus prievado konfigūracinių parametrų nustatymas, kadangi netinkamai pasirinkus vertes, duomenys nebus perduodami. Ši sąsaja yra sudaroma atskiro lango pavidalu, todėl jos kodas yra atskirtas nuo pagrindinės kodo dalies.

1.1. Parametrų parinkimo langas



2 pav. Parametrų nustatymo langas

```
#UART settings options
port_options = []
stop_bits_options = [1, 2]
baud_rate_options = [9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600]
data_bits_options = [5, 6, 7, 8]
```

Nustatymų langui reikalingos pasirinkimų vertės, kurias apsisąšome masyve. „Apply“ mygtuku reikalinga išsaugoti pasirinktas vertes, kaip kintamuosius, kurie bus toliau naudojami „serial“ paketo, norint atidaryti pasirinktą prievadą.

```
screen_width = myWindow_uart_settings.wininfo_screenwidth()
screen_height = myWindow_uart_settings.wininfo_screenheight()
x_cordinate = int((screen_width/2) - (WINDOW_WIDTH/2))
y_cordinate = int((screen_height/2) - (WINDOW_HEIGHT/2))
myWindow_uart_settings.geometry("{}x{}+{}+{}".format(WINDOW_WIDTH, WINDOW_HEIGHT,
x_cordinate, y_cordinate))
```

Su šiomis eilutėmis yra apskaičiuojamas monitoriaus dydis, surandamos centrinės koordinatės ir yra suformatuojamos į funkcija „geometry“, kuriai reikalinga perduoti „string“ tipo kintamąjį formatu „widthxheight+x+y“[1]. Tai atliekama pasinaudojant funkcija, tačiau galima ir naudotat specialų „F-Strings“ formatavimą, kuris yra implementuotas „python“ kalboje. Mygtukas „apply“ yra kaip pertrauktis vykdyti funkciją, pateiktą žemiau:

```
def com_port_update_settings(window,port, baud, parity, stop):
    #update globals for main GUI
    KK_Lab2_GLOBALS.selected_baud_rate = baud
    KK_Lab2_GLOBALS.selected_port = port
    KK_Lab2_GLOBALS.selected_parity = parity
    KK_Lab2_GLOBALS.selected_stop_bits = stop
    window.destroy()
```

Šioje funkcijoje yra išsaugomi globalūs kintamieji, kurie yra kitame faile, siekiant palaikyti ryšį tarp atskirų „source“ failų. Lango išjungimas vyksta langui kreipiantis į „destroy“ funkcija. Nuoseklios komunikacijos galimi prievadai yra gaunami pasinaudojus jų suteikta komanda ir apsirašius funkcijoje[2]:

```
def com_port_list_update(port_options):
```

```
    port_options = serial.tools.list_ports.comports()
```

Ši funkcija yra kviečiama kaskart įjungus langą. Toliau yra apibrėžiamos visi grafinėi vartotojo sąsajai reikalingi valdikliai („widgets“). Po to yra nustatomos visos atvaizduojamos vertės tik atidarius langą, sudaromas pasirinktas tinklėlis ir valdikliai yra įtraukiami į langą.

```
    #update port list
    com_port_list_update(port_options)

    myLabel_port_name = tk.Label(myWindow_uart_settings, text="Port name",
font=KK_Lab2_GLOBALS.myFontMain,justify="center",padx=50)
    myComboBox_port_name = ttk.Combobox(myWindow_uart_settings,
state='readonly', font=KK_Lab2_GLOBALS.myFontMain,width=15)
    myLabel_port_rate = tk.Label(myWindow_uart_settings, text="Baud rate",
font=KK_Lab2_GLOBALS.myFontMain,justify="center",padx=50)
    myComboBox_port_rate = ttk.Combobox(myWindow_uart_settings,
state='readonly', font=KK_Lab2_GLOBALS.myFontMain,width=15)
    myLabel_port_parity = tk.Label(myWindow_uart_settings, text="Parity",
font=KK_Lab2_GLOBALS.myFontMain,justify="center",padx=50)
    myComboBox_port_parity = ttk.Combobox(myWindow_uart_settings,
state='readonly', font=KK_Lab2_GLOBALS.myFontMain,width=15)
    myLabel_port_stop_bits = tk.Label(myWindow_uart_settings, text="Stop bits",
font=KK_Lab2_GLOBALS.myFontMain,justify="center",padx=50)
    myComboBox_port_stop_bits = ttk.Combobox(myWindow_uart_settings,
state='readonly', font=KK_Lab2_GLOBALS.myFontMain,width=15)
    myLabel_port_data_bits = tk.Label(myWindow_uart_settings, text="Data bits",
font=KK_Lab2_GLOBALS.myFontMain,justify="center",padx=50)
    myComboBox_port_data_bits = ttk.Combobox(myWindow_uart_settings,
state='readonly', font=KK_Lab2_GLOBALS.myFontMain,width=15)

    myButton_port_settings_apply = tk.Button(myWindow_uart_settings, text="Apply", command=lambda
:com_port_update_settings(myWindow_uart_settings,myComboBox_port_name.get(),myComboBox_port_rate.
get(),myComboBox_port_parity.get(),myComboBox_port_stop_bits.get()),width=4,
height=1,font=KK_Lab2_GLOBALS.myFontMain)
    myButton_port_settings_cancel = tk.Button(myWindow_uart_settings, text="Cancel",
command=myWindow_uart_settings.destroy,width=4, height=1,font=KK_Lab2_GLOBALS.myFontMain)
```

```

myComboBox_port_name['values'] = [port.device for port in serial.tools.list_ports.comports()]
myComboBox_port_name.set("Select COM port")
myComboBox_port_rate['values'] = baud_rate_options
myComboBox_port_rate.set(KK_Lab2_GLOBALS.selected_baud_rate)
myComboBox_port_parity['values'] = list(KK_Lab2_GLOBALS.parity_option_dict.keys())
myComboBox_port_parity.set(KK_Lab2_GLOBALS.selected_parity)
myComboBox_port_stop_bits['values'] = stop_bits_options
myComboBox_port_stop_bits.set(KK_Lab2_GLOBALS.selected_stop_bits)
myComboBox_port_data_bits['values'] = data_bits_options
myComboBox_port_data_bits.set(KK_Lab2_GLOBALS.selected_data_bits)

myLabel_port_name.grid(padx=WINDOW_WIDTH/2-75, row=1, column=0,columnspan=2)
myComboBox_port_name.grid(padx=WINDOW_WIDTH/2-75, row=2, column=0,columnspan=2)
myLabel_port_rate.grid(padx=WINDOW_WIDTH/2-75, row=4, column=0,columnspan=2)
myComboBox_port_rate.grid(padx=WINDOW_WIDTH/2-75, row=5, column=0,columnspan=2)
myLabel_port_parity.grid(padx=WINDOW_WIDTH/2-75, row=7, column=0,columnspan=2)
myComboBox_port_parity.grid(padx=WINDOW_WIDTH/2-75, row=8, column=0,columnspan=2)
myLabel_port_stop_bits.grid(padx=WINDOW_WIDTH/2-75, row=10, column=0,columnspan=2)
myComboBox_port_stop_bits.grid(padx=WINDOW_WIDTH/2-75, row=11, column=0,columnspan=2)

#add spacing between comboboxes and buttons
tk.Label(myWindow_uart_settings, text="",justify="center").grid(row=3, column=0)
tk.Label(myWindow_uart_settings, text="",justify="center").grid(row=6, column=0)
tk.Label(myWindow_uart_settings, text="",justify="center").grid(row=9, column=0)
tk.Label(myWindow_uart_settings, text="",justify="center").grid(row=12, column=0)

myButton_port_settings_apply.grid(row=13, column=0,ipadx=20)
myButton_port_settings_cancel.grid(row=13, column=1,ipadx=20)

```

paskutiniai „label“ valdikliai yra tuščio teksto, jie yra naudojami tik tam, jog tarp tinklelio būtų sudarytas tarpas, nes „tkinter“ visus neapibrėžtus tinklelius sutraukia, apjungia.

1.2. Globalių kintamųjų failas

Globaliems kintamiesiems perduoti yra naudojamas atskiras „.py“ failas:

```

# Default settings and global variables
import serial
import numpy as np

Port_connected = False
Clearing_logs = False
serialPort = serial.Serial()

MYFONTMAIN = ("Times New Roman", 12)
MYFONTBIGGEST = ("Times New Roman", 18,'bold')
MYFONTMID = ("Times New Roman", 15,'bold')

myGraphIndex = 0 #index for graph values
Index_gps = 0
myGraphIndexArraySensor = []

```

```

myGraphIndexArrayGPS = []
GRAPH_SHIFT_SIZE = 60 # 60 latest values

Temp_array = []
Press_array = []
Time_array = []
Lat_array = []
Lon_array = []

selected_port = " "
selected_baud_rate = 9600
selected_parity = "None"
selected_stop_bits = 1
selected_data_bits = 8

parity_option_dict = {"None": serial.PARITY_NONE, "Even": serial.PARITY_EVEN, "Odd":
serial.PARITY_ODD, "Mark": serial.PARITY_MARK, "Space": serial.PARITY_SPACE}

```

Matplotlib palaiko skirtingus šriftus, todėl naudojami keli dydžiai, kurie yra apsašomi kaip globalūs. Dėl programos saugumo yra naudojami du kintamieji, kurių vienas iš jų nusako ar yra prijungtas prievadas prie „serial“ paketo. Kitas kintamasis yra duomenų išvalymui reikalingas, kad trinant duomenis nebūtų ant viršaus užrašoma, taip išvalymas lenktyniautu su kodo įrašymu. Lyginumas („parity“) yra aprašomas „string“ tipo masyvu, tačiau į „serial“ paketą reikalinga duomenis perduoti kitokiu tipu, todėl yra sudaromas žodynas („dictionary“), kuris vertes priskiria kintamiesiems. Taip galima iššifravus tekstą nusakyti, kokio lyginumo reikalaujama pasinaudojant žodyno „keys“ ir „get“ funkcijomis.

1.3. Funkcijų failas

Kadangi „python“ programavimo kalba neturi funkcijoms deklaravimo, todėl funkcijas galima tik importuoti į pagrindinį failą, kuriame yra visas langas. Kintamieji, kurie yra naudojami funkcijose, gali būti pasirenkami, tačiau tam reikalingas tik vienas failas, kuris yra naudojamas kaip atskirai globaliems kintamiesiems apsašyti. Norint naudoti funkcijas, reikalinga iš pagrindinio failo išsiimti sukurtus valdiklius ir juos konfigūruoti. Kai funkcija yra toje pačioje vietoje, kaip ir pagrindinis failas, tai problemų nesukelia, tačiau kai naudojamas funkcijų failas aprašyti veiksmus, kuriuose yra ir iš pagrindinio failo esantys kintamieji, negalima juos importuoti, nes pats funkcijų failas bus importuojamas pagrindiniame faile, taip gaunamas rekursinis kvietimas („circular dependancy“). Vienas iš sprendimo būdų yra į funkcijas paduoti visus kintamuosius, kurie bus naudojami funkcijoje, kas šiuo atveju ir yra daroma. Dideliems projektams šis metodas nėra geras, nes jeigu funkcija gaunasi didelė, reikalinga daug kintamųjų perdavimo, taip galima įvesti daugiau vartotojo klaidų („user error“).

```

def
COM_port_Open_Close(button,label,root,textbox,fix,time,satellites,HDOP,TEMP_AXIS,PRESS_AXIS,GRAPH
,MAP,LAT_VALUE,LON_VALUE):
    if globals.Port_connected == False:
        try:
            globals.serialPort.baudrate = globals.selected_baud_rate
            globals.serialPort.port = globals.selected_port

```

```

globals.serialPort.parity = globals.parity_option_dict.get(globals.selected_parity)
globals.serialPort.stopbits = int(globals.selected_stop_bits)
globals.serialPort.bytesize = int(globals.selected_data_bits)
globals.serialPort.timeout = 0.01 #10 ms

globals.serialPort.close()
globals.serialPort.open()
button['text'] = "Close port"
label['background'] = "green"
label['text'] = "OPEN"
globals.Port_connected = True
root.after(10,
lambda:COM_port_read_data(root,textbox,fix,time,satellites,HDOP,TEMP_AXIS,PRESS_AXIS,GRAPH,MAP,LA
T_VALUE,LON_VALUE))#start reading
    except Exception as e:
        tk.messagebox.showerror("ERROR", f"Port can't be opened: {e}")
        label['background'] = "red"
        label['text'] = "CLOSED"
    else:
        try:
            root.after_cancel(COM_port_read_data)
            globals.serialPort.close()
            button['text'] = "Open port"
            label['background'] = "red"
            label['text'] = "CLOSED"
            fix['background'] = "red"
            time['text'] = "---:--:--"
            satellites['text'] = "--"
            HDOP['text'] = "--"
            LAT_VALUE['text'] = "--"
            LON_VALUE['text'] = "--"
            globals.Port_connected = False
        except Exception as e:
            messagebox.showerror("ERROR", f"Port can't be closed: {e}")

```

Prievado atidarymui naudojama vėliavėlė, atidarant tik tada, kai procesas nevyksta. Python turi klaidų kontrolę, todėl yra naudojamas “try except” sąlygos, kurios atlieka kodą tik tada, jeigu proceso metu nebuvo jokių klaidų, kurios yra aprašomos viduje funkcijų kaip grįžamosios vertės (“return”). “Python” turi “casting” ypatybę, kuri naudojama vieno tipo kintamiesiems paversti į kito tipo kintamuosius. Iškviesta funkcija “messagebox” yra grafinis langas, kuris iššoka kaip langas, kuriuo galima pranešti vartotoją.

```

def com_port_log_clear(textbox,temp_graph,press_graph,canvas,map):#clear logs,graph and index

    textbox['state'] = 'normal'
    textbox.delete(1.0, tk.END)
    textbox['state'] = 'disabled'
    globals.Clearing_logs = True
    globals.myGraphIndex = 0
    globals.Index_gps = 0
    globals.Press_array = []

```

```

globals.Temp_array = []
globals.myGraphIndexArraySensor = []
globals.myGraphIndexArrayGPS = []
globals.Lat_array = []
globals.Lon_array = []
globals.Time_array = []
map.delete_all_marker()
globals.Clearing_logs = False
canvas.draw()
globals.serialPort.reset_input_buffer()

def
COM_port_read_data(root,textbox,fix,time,satellites,HDOP,TEMP_AXIS,PRESS_AXIS,GRAPH,MAP,LAT_VALUE
,LON_VALUE):
    if globals.Port_connected:
        if globals.serialPort.in_waiting:
            try:
                data = globals.serialPort.readline()
                if data:
                    string_data = data.decode('ascii')
                    com_port_log_data(string_data,textbox)

com_port_display_data(string_data,fix,time,satellites,HDOP,TEMP_AXIS,PRESS_AXIS,GRAPH,MAP,LAT_VAL
UE,LON_VALUE)
            except Exception as e:
                log_msg = f"Error reading data: {e}\n"
                com_port_log_data(log_msg,textbox)
            root.after(10, lambda:
COM_port_read_data(root,textbox,fix,time,satellites,HDOP,TEMP_AXIS,PRESS_AXIS,GRAPH,MAP,LAT_VALUE
,LON_VALUE))#blocking every 10 ms while port open
            else: root.after_cancel(COM_port_read_data)
Paketa "serial.readline" skaito iki "n" naujos linijos, kuri mūsų atveju atitinka paketo pabaigą, todėl
ši galima naudoti paprastam duomenų apdorojimui. Išgavus tekstą šis yra dekoduojamas "ascii"
format ir perduodamas į išvedimą į tekstinį langą bei atiduodamas atvaizdavimui kitiems langams.

def com_port_log_data(data,textbox):
    textbox['state'] = 'normal'
    try:
        textbox.insert(tk.END, data)
    except:
        textbox.insert(tk.END, data)
    textbox.see(tk.END)
    textbox['state'] = 'disabled'
Šioje funkcijoje yra įjungiamas teksto rašymas į tekstinį langą ir įterpiami duomenys, pradedant nuo
galo, taip neišnaikinant to, kas jau yra įrašyta.

def
com_port_display_data(data,fix,time,satellites,HDOP,TEMP_AXIS,PRESS_AXIS,CANVAS,MAP,LAT_VALUE,LON
_VALUE):
    if data.startswith("$PNLBLEPS"):

```

```

#command, pressure 1.1f, tempeature 1.1f, checksum
splitted_data = data.split(',')
pressure = float(splitted_data[1])
temperature = float(splitted_data[2])
checksum = splitted_data[3].strip() #remove \r\n end

globals.Temp_array.append(temperature)
globals.Press_array.append(pressure)
globals.myGraphIndexArraySensor.append(globals.myGraphIndex)

globals.Temp_array = globals.Temp_array[-(3*globals.GRAPH_SHIFT_SIZE):]
globals.Press_array = globals.Press_array[-(3*globals.GRAPH_SHIFT_SIZE):]
globals.myGraphIndexArraySensor = globals.myGraphIndexArraySensor[-
(3*globals.GRAPH_SHIFT_SIZE):]

TEMP_AXIS.set_data(globals.myGraphIndexArraySensor,globals.Temp_array)
PRESS_AXIS.set_data(globals.myGraphIndexArraySensor,globals.Press_array)

TEMP_AXIS.axes.relim(); TEMP_AXIS.axes.autoscale_view()
PRESS_AXIS.axes.relim(); PRESS_AXIS.axes.autoscale_view()
globals.myGraphIndex +=1
CANVAS.draw_idle()

#fix,latitude(DMS),NS,longtitude(DMS),EW,UTC+3,satellites,HDOP,checksum
elif data.startswith("$PNLBGPS"):
    splitted_data = data.split(',')
    GNSS_fix = splitted_data[1]
    latitude = splitted_data[2]
    NS = splitted_data[3]
    longitude = splitted_data[4]
    EW = splitted_data[5]
    EET_summer = splitted_data[6]
    satellite_count = splitted_data[7]
    HDOP_var = splitted_data[8]
    checksum = splitted_data[9].split()#remove \r\n end

    globals.Lat_array.append(float(latitude) if NS == 'N' else -float(latitude))
    globals.Lon_array.append(float(longtitude) if EW == 'E' else -float(longtitude))
    globals.Time_array.append(EET_summer)
    globals.myGraphIndexArrayGPS.append(globals.Index_gps)

    globals.Time_array = globals.Time_array[-globals.GRAPH_SHIFT_SIZE:]
    globals.Lat_array = globals.Lat_array[-globals.GRAPH_SHIFT_SIZE:]
    globals.Lon_array = globals.Lon_array[-globals.GRAPH_SHIFT_SIZE:]
    globals.myGraphIndexArrayGPS = globals.myGraphIndexArrayGPS[-globals.GRAPH_SHIFT_SIZE:]

    if globals.Clearing_logs == False:
        MAP.set_marker(float(latitude),float(longtitude))
    fix['background'] = "green" if int(GNSS_fix) == 1 else "red"
    time['text'] = EET_summer[0:2]+':'+EET_summer[2:4]+':'+EET_summer[4:6]

```

```

satellites['text'] = satellite_count
HDOP['text'] = HDOP_var
LAT_VALUE['text'] = f"{float(latitude) if NS == 'N' else -float(latitude):.5f}"
LON_VALUE['text'] = f"{float(longtitude) if EW == 'E' else -float(longtitude):.5f}"
globals.Index_gps +=1

```

Pradžioje tikriname priimtą paketą ir priklausomai nuo to apdorojame. Apdorojus yra perduodama į globalius kintamuosius reikalingi duomenys.[3]

```

globals.Temp_array = globals.Temp_array[-globals.Graph_shift_size:]
globals.Press_array = globals.Press_array[-globals.Graph_shift_size:]
globals.myGraphIndexArray = globals.myGraphIndexArray[-globals.GRAPH_SHIFT_SIZE:]

```

Čia yra sudarytas slenkantis buferis(“first in last out”), kurio ilgis yra nurodytas prie globalių kintamųjų. Simbolis “:” atskiria pradžią ir pabaigą. Pabaiga šiuo atveju yra nenusakyta, todėl imama vertė kaip paskutinis elementas, o pradžia yra neigiama dydžiui. Tuo atveju, jeigu verčių yra daugiau nei nurodytas neigiamas kintamasis, visos vertės yra paaimamos, taip gauname paprastai implementuotą slenkantį buferį.

Matplotlib funkcija “draw_idle” yra naudojama vietoj “draw” todėl, nes yra greitesnis sprendimas atvaizdavimui, kadangi neleidžia užblokuoti pertraukčių ir atvaizdavimas vyksta tik tuo atveju, jeigu yra pasikeitimas, o ne visada iškvietus funkciją. „GNSS FIX“ yra indikacija, kuri grafiškai nusako, ar GNSS modulis fiksavo savo poziciją, taip galima vizualiai matyti ar gaunami duomenys turi reikšmę. Laikas yra formuojamas XX:XX:XX pavidalu, todėl reikalinga vertės išskirti.

1.4. Pagrindinis failas

Pagrindiniame faile yra inicializuojami visi valdikliai, kurie yra įtraukiami į grafinį langą tinkliniu pavidalu. „matplotlib“ sukurtos figūros yra coliais, tačiau galima pakeisti į pikselius, taip turint aiškius ruožus, reikalingus grafinės vartotojo sąsajos sukūrimui, kadangi dauguma kitų valdiklių „tkinter“ paketo yra pikselių pavidalu.

```

#graph matlab
px = 1/matplotlib.rcParams['figure.dpi'] # pixel in inches
myFigureMain =
Figure(dpi=100,tight_layout=True,figsize=(WIDTH_RIGHT_GUI*px,HEIGHT_RIGHT_GUI/2*px))
myGraphPressureAx = myFigureMain.add_subplot(212)
myGraphTempAx = myFigureMain.add_subplot(211)

```

```

myGraphTemp, = myGraphTempAx.plot([],[])
myGraphPres, = myGraphPressureAx.plot([],[])

```

LPS22HB jutiklis išduoda temperatūrą ir slėgį, todėl šiuos duomenis ir atvaizduojame, tačiau vizualizavimo greitaveikai yra sudaromas „plot“ failo tipas ir jis yra užkraunamas programos metu, o po to duomenys yra įtraukiami su funkcijomis „set_data“, taip neatliekant papildomų skaičiavimų kiekvieną kartą. Iš pradžių buvo naudojama „scatter“ atvaizdavimo tipas, tačiau šis neturi “set_data” funkcijos, kuri įtraukia tik papildomas vertes, o naudojant „scatter“ funkciją kiekvieną kartą net ir ne visus duomenis vaizduojant programa pradėdavo lėtėti.

```

matplotlib.rcParams.update({'axes.titlesize':12, 'figure.titlesize':18})
myFigureMain.suptitle('LPS22HB data')
myGraphTempAx.set_title('Temperature graph')

```

```

myGraphPressureAx.set_title('Pressure graph')
myGraphTempAx.set_ylabel(r"Temperature, $\degree$C")
myGraphPressureAx.set_ylabel("Pressure, hPa")
myGraphPressureAx.set_xlabel("Samples")
myGraphTempAx.grid()
myGraphPressureAx.grid()
myGraphTempAx.set_xlabel("Samples")
myGraphPressureAx.ticklabel_format(style='plain',useOffset=False)
myGraphPressureAx.set_xlim(auto=True)
myGraphTempAx.set_xlim(auto=True)

```

Šiame fragmente yra nustatomi grafiko duomenys inicializacijos metu.

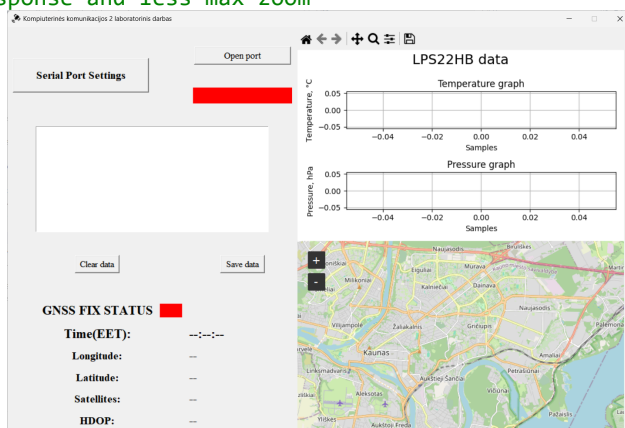
Kadangi yra naudojamas tinklinis išdėstymas, o grafikai užima didelę dalį vietos, yra sukuriama atskiri rėmai, kurie užima kelias tinklo pozicijas, taip galime paprasčiau išdėstyti didesnes formas. Iš pirmo yra sudaromas dešinėje pusėje esantis rėmas, kuriame yra papildomai sudaryti du rėmai, viršutinis yra priskiriamas „matplotlib“ paketo sudarytam grafikui, o apatiniam yra naudojamas „tkinterMapView“ paketas, kuris gali atvaizduoti įvairių serverių ZXY taškinės matricos duomenis (naudojamas nemokamos prieigos serveris <https://tile.openstreetmap.org/{z}/{x}/{y}.png>). Šiame yra trys koordinatės, „zoom“ x ir y, kurias įvedus gaunamas .png failas ir yra importuojamas į grafinę vartotojo sąsają. Kadangi duomenys buvo siunčiami iš mikroprocesoriaus DMS formatu koordinatėms atvaizduoti, šis buvo nepatogus įterpti į grafinį langą koordinatės, taip pat jos būdavo didesnės paklaidos, todėl mikroprocesorius buvo perprogramuojamas ir koordinatės siuntimas pakeistas į „Decimal Degrees“ arba kitaip „DD“ formatą, kurį palaiko grafinis žemėlapis, pateikiant koordinatės.

```

#right side gui
myToolbar = NavigationToolbar2Tk(myCanvas, myFrame_right_side,pack_toolbar=False)
myCanvas.draw()
myToolbar.update()

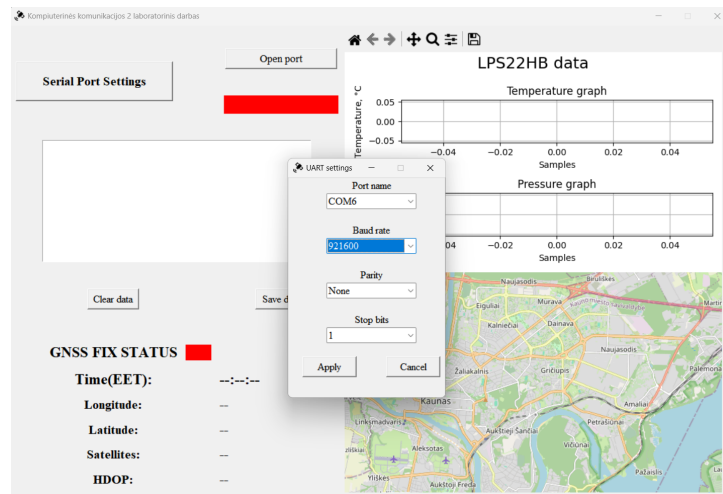
myMapWidget = tkintermapview.TkinterMapView(myFrame_right_bottom, width=WIDTH_RIGHT_GUI,
height=HEIGHT_RIGHT_GUI/2,)
myMapWidget.set_position(54.90396923101469, 23.957806638243493) #KTU 11 rumai
myMapWidget.set_zoom(12)
tile_server_url = "https://tile.openstreetmap.org/{z}/{x}/{y}.png"
myMapWidget.set_tile_server(tile_server_url,max_zoom = 17)
#light map for faster response and less max zoom

```

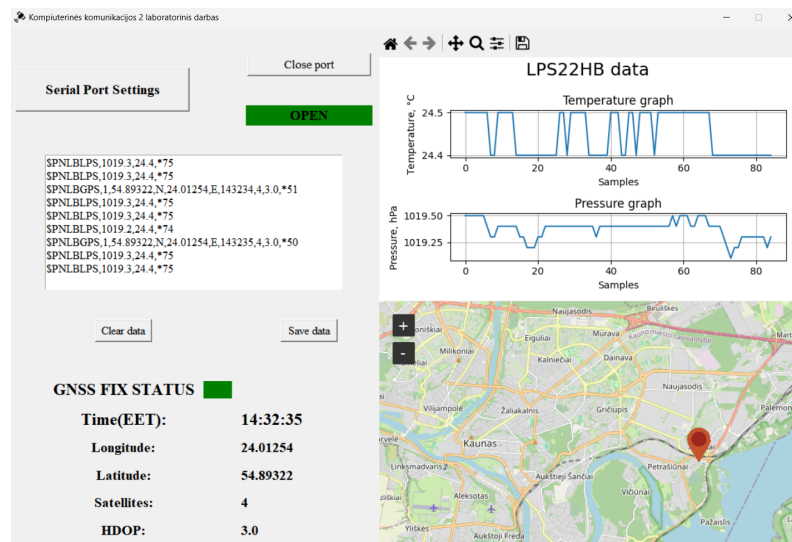


3 pav. Grafinės vartotojo sąsajos pradinis vaizdas

Po prievado mygtuku esantis statusas nusako, ar yra atidarytas prievadas, kuris užsidega žaliai arba raudonai. Šiuo principu yra nurodomas GNSS modulio fiksavimas, kadangi šiam atvaizduoti yra tik nurodomas 0 bei 1.

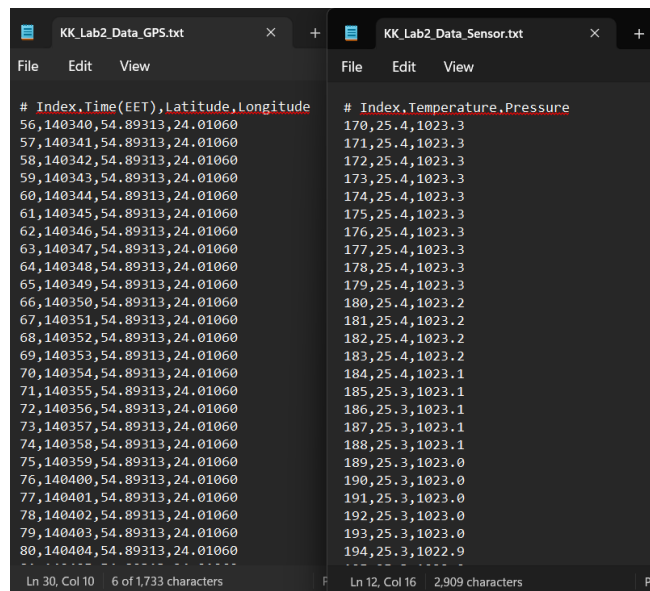


4 pav. Nuoseklaus prievado iššokantis konfigūravimo langas



5 pav. Duomenų vizualizavimas vartotojo sąsajoje

Duomenys yra išsaugojami „Save Data“ mygtuku į du failus. Pasirinkta naudoti du failus todėl, nes duomenys yra skirtingo periodo, jutiklių duomenys yra gaunami kas 1/3 sekundės, todėl juos būtų nepatogu atvaizduoti „csv“ pavidalu. Saugojami yra paskutinės minutės duomenys, todėl reikalinga 60 verčių GPS signalo ir 180 jutiklio duomenų.



6 pav. Duomenų išsaugojimas tekstiniame faile „csv“ formatu

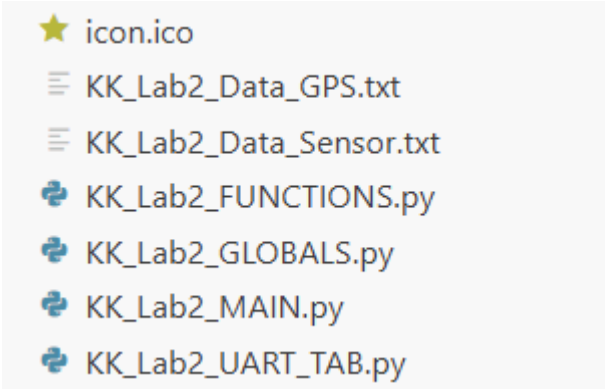
Duomenų saugojimas yra pasinaudojant „numpy“ funkcijomis[4].

```
def Save_plot_data():
    if len(globals.Press_array) > 0 and len(globals.Temp_array) > 0 and len(globals.Time_array) > 0: # GPS and LPS data not empty
        try:
            #format a big array
            data1 = rfn.merge_arrays((np.array(globals.myGraphIndexArraySensor),
np.array(globals.Temp_array), np.array(globals.Press_array)))
            np.savetxt('KK_Lab2_Data_Sensor.txt', data1, fmt=["%d", "%.1f", "%.1f"],
delimiter=',', header="Index, Temperature, Pressure")
            data2 =
            rfn.merge_arrays((np.array(globals.myGraphIndexArrayGPS), np.array(globals.Time_array),
np.array(globals.Lat_array), np.array(globals.Lon_array)))
            np.savetxt('KK_Lab2_Data_GPS.txt', data2, fmt=["%d", "%s", "%.5f", "%.5f"],
delimiter=',', header="Index, Time(EET), Latitude, Longitude")
            messagebox.showinfo("INFO", "Data saved to KK_Lab2_Data_Sensor.txt and
KK_Lab2_Data_GPS.txt")
        except Exception as e:
            messagebox.showerror("ERROR", f"Error saving data: {e}")
    else:
        messagebox.showwarning("WARNING", "No data to save.")
```

Masyvų manipuliacijai reikalinga funkcija, kuri yra pasiimta iš numpy paketo:

```
from numpy.lib import recfunctions as rfn
```

Kadangi duomenys yra skirtingų kintamųjų tipų, reikalinga kiekvieną eilutę aprašyti, taip sudarant struktūrą, panašią į duomenų bazės. Tai yra atliekama „merge_arrays“ funkcija, kurioje yra suformuojami masyvai, kurie turi būti „np.array“ tipo, tačiau yra kaip „list“ todėl daroma konversija kiekvienam masyvui.



- ★ icon.ico
- ☰ KK_Lab2_Data_GPS.txt
- ☰ KK_Lab2_Data_Sensor.txt
- 🔗 KK_Lab2_FUNCTIONS.py
- 🔗 KK_Lab2_GLOBALS.py
- 🔗 KK_Lab2_MAIN.py
- 🔗 KK_Lab2_UART_TAB.py

7 pav. Programos metu naudojami failai

Išvados

Duomenys, gauti per protokolą, yra apdorojami ir atvaizduojami įvairiais būdais: tekstiniu pavidalu, grafiniu pavidalu, žemėlapyje, „csv“ suformuotame tekstiniame faile, indikacijomis. Nuoseklaus prievado nustatymams yra sudarytas atskiras langas, kuris programą paverčia paprastesnę vartotojui naudoti. Grafiškai atvaizduojamiems jutiklio duomenims yra įtraukta įrankių juosta, taip leidžiant grafiką išsaugoti „png“ formatu.

Šaltiniai

- [1] crispengari, „Answer to ‚How to center a window on the screen in Tkinter?‘“, Stack Overflow. Žiūrėta: 2025 m. spalio 11 d. [Interaktyvus]. Adresas: <https://stackoverflow.com/a/63848196>
- [2] „pySerial API — pySerial 3.4 documentation“. Žiūrėta: 2025 m. spalio 11 d. [Interaktyvus]. Adresas: https://pyserial.readthedocs.io/en/latest/pyserial_api.html
- [3] JayRedditDev, „How to parse serial data (byte to string?)“, r/learnpython. Žiūrėta: 2025 m. spalio 11 d. [Interaktyvus]. Adresas: https://www.reddit.com/r/learnpython/comments/fnmnok/how_to_parse_serial_data_byte_to_string
- [4] „numpy.savetxt — NumPy v2.3 Manual“. Žiūrėta: 2025 m. spalio 11 d. [Interaktyvus]. Adresas: <https://numpy.org/doc/stable/reference/generated/numpy.savetxt.html>